

Final Report: Retail Data Lakehouse Capstone

This report summarizes the architecture, cost considerations, and a short reflection for the retail data lakehouse capstone project.

Architecture Overview

The solution uses a medallion architecture that moves data from raw S3 storage through Bronze, Silver, and Gold layers for analytics and reporting.

ARCHITECTURE.md

Data Lake Architecture (Medallion Architecture)

Overview

This solution follows the Medallion Architecture pattern, organizing data into three logical zones: **Bronze (Raw)**, **Silver (Curated)**, and **Gold (Consumption)**. All data is stored in a single Amazon S3 bucket using top-level prefixes to separate the layers and support scalable data processing.

```
s3://company-datalake/
├── raw/           # Bronze Layer
├── curated/       # Silver Layer
└── consumption/  # Gold Layer
```

1. Bronze Layer (Raw Zone)

The Bronze layer stores data exactly as received from source systems with minimal or no transformations. It serves as the historical source of truth for auditing, replay, and recovery purposes.

Datasets

```
raw/
├── customers/
├── products/
└── orders/
    ├── dt=2026-07-06/
    └── dt=2026-07-07/
```

Partitioning Convention

Data in the orders dataset is partitioned by ingestion date:

```
raw/orders/dt=YYYY-MM-DD/
```

Examples:

- raw/orders/dt=2026-07-06/
- raw/orders/dt=2026-07-07/

This partitioning strategy improves query performance and enables incremental processing.

Naming and Partitioning Standards

S3 Prefix Naming

- Layer names use lowercase:
 - raw/
 - curated/
 - consumption/
- Dataset names use lowercase and singular/plural business names:
 - customers
 - orders
 - products

2. Silver Layer (Curated Zone)

The Silver layer contains cleaned, validated, and standardized datasets derived from the Bronze layer. Common transformations include:

- Data quality validation
- Removal of duplicates
- Standardization of formats
- Schema enforcement
- Basic business rule application

Example structure:

```
curated/
├── customers/
├── products/
└── orders/
```

The Silver layer provides trusted datasets for downstream analytics and business reporting.

3. Gold Layer (Consumption Zone)

The Gold layer contains business-ready datasets optimized for reporting, dashboards, and analytics use cases.

Typical contents include:

- Aggregated metrics
- KPI datasets
- Dimension and fact tables
- Reporting views

Example structure:

```
consumption/
├── sales_summary/
├── customer_metrics/
└── product_performance/
```

This layer is designed for direct consumption by BI tools and business users.

Principles

- Single S3 bucket with three top-level prefixes (Bronze, Silver, Gold)
- Consistent naming conventions
- Date-based partitioning for incremental ingestion
- Separation of raw, curated, and consumption-ready data
- Supports scalability, maintainability, and governance

Architecture Pattern: Medallion Architecture (Bronze → Silver → Gold) | Storage Platform: Amazon S3 | Partition Strategy: Dataset-level folders with date-based partitions for incremental ingestion.

Flow:

1. S3 Buckets creation
2. Datalake Ingestion
3. Bronze / Silver / Gold Layers
4. Glue Catalog / Lake Formation
5. Athena / Dashboards

Cost Review

This capstone pipeline was implemented as a lightweight, hands-on lakehouse workflow rather than a fully hardened production platform. I provisioned a small S3 landing zone for raw and curated data, a Glue Data Catalog database and crawlers, Lake Formation permissions for governance, and a short-lived Databricks environment for Bronze, Silver, and Gold processing. I also used Athena for basic SQL validation and a small budget alert to keep the exercise under control. The goal was to demonstrate the end-to-end pattern, not to optimize every cost knob from day one.

What I provisioned

- S3 buckets and folders for raw data, checkpoints, and curated Delta outputs.
- Glue crawlers and a Data Catalog database to register raw tables and infer schemas.
- Lake Formation permissions and tags to control access to governed data.
- A Databricks notebook-based pipeline and a small job cluster for ETL execution.
- Athena SQL queries for validation and reporting checks.

What I tore down

For the exercise, I removed or minimized anything that was only needed for experimentation. That included temporary test clusters, unused exploratory storage paths, and any non-essential access patterns that were created while validating the pipeline. The core data assets and governance setup were kept in place so the workflow could still be explained and demonstrated clearly. In a real environment, I would keep the persistent storage and catalog metadata but retire temporary compute resources immediately after job completion to avoid idle spend.

Estimated monthly cost at 10x data volume

Using rough public-cloud pricing assumptions, the current setup would likely cost only a few hundred dollars per month for the small-scale workload used in this capstone. A reasonable estimate would be about \$50-\$80 for S3 storage, \$10-\$20 for Glue and crawler activity, \$10-\$20 for Athena scan charges, and \$200-\$400 for Databricks compute if the job cluster runs for a few hours each day. At 10x the data volume, the monthly cost would increase to roughly \$600-\$900 per month, with Databricks compute and Athena scanning becoming the dominant cost drivers. The larger data volume would also increase the need for partitioning, incremental processing, and better orchestration so the workload stays efficient instead of scaling linearly.

Reflection

If this were a real production system, I would change one major design choice: I would replace the single shared notebook-driven approach with a more formal orchestration pattern using autoscaled, job-scoped Databricks clusters plus incremental Delta processing. The reason is simple: a production pipeline needs predictable performance, lower idle compute cost, and better operational reliability. In the current capstone setup, the workflow is easy to explain and quick to run, but in production it would be too fragile and too expensive to depend on ad hoc notebook execution for recurring data loads.